



Санкт-Петербургский государственный университет
Кафедра системного программирования

Сравнение производительности алгоритмов на графах из SPLA/GraphBLAS+LAGraph

Гавриленко Михаил, группа 23.Б15-мм

Санкт-Петербург
2023

Постановка задачи

Цель: исследовать алгоритмы BFS, SSSP, TC, PR из библиотек SPLA и GraphBLAS/LAGraph

Задачи:

- Сформулировать гипотезы о производительности
- Произвести корректные замеры: правильный источник обхода, учёт JIT-разогрева GPU
- Исследовать особенности: поддержку ориентированных/неориентированных графов, влияние топологии на скорость
- Сделать выводы о гипотезах

Детали экспериментов

Методология:

- Источник обхода: вершина с максимальной исходящей степенью (стандарт LAGraph bench suite)
- Прогревочный запуск отброшен (JIT OpenCL для SPLA, холодный кэш для LAGraph)
- Медиана по 14 измерениям (итерации 2–15)
- Профилирование через GxB_BURBLE и Instruments (Xcode)

Конфигурация системы:

- M2 Max (arm64): 12 CPU ядер (8P+4E), 38 GPU ядер @ 1000 МГц, 32 ГБ unified memory
- OpenCL 1.2, clang 17.0.0, OMP_NUM_THREADS=10

Датасеты:

- soc-LiveJournal1 (68M пёбер, social), patents (15M, citation)
- wiki-Talk (5M, communication), rgg_n_2_22_s0 (30M, geometric random)
- roadNet-CA (2.7M, road) — для TC

BFS: первичные тезисы

- GPU эффективнее на социальных графах (широкий фронтир, малый диаметр)
- На геометрических случайных графах (RGG) CPU выигрывает — высокий диаметр, многократные запуски ядер с низкой загрузкой GPU
- LAGraph BFS на направленных графах требует кэша транспонированной матрицы (LAGraph_Cached_AT) для включения режима push+pull
- Первый запуск SPLA на GPU: 10–30× медленнее из-за JIT-компиляции ядер OpenCL

BFS: поддержка направленных/ненаправленных графов

LAGr_BreadthFirstSearch

Поддерживает оба режима через LAGraph_Kind:

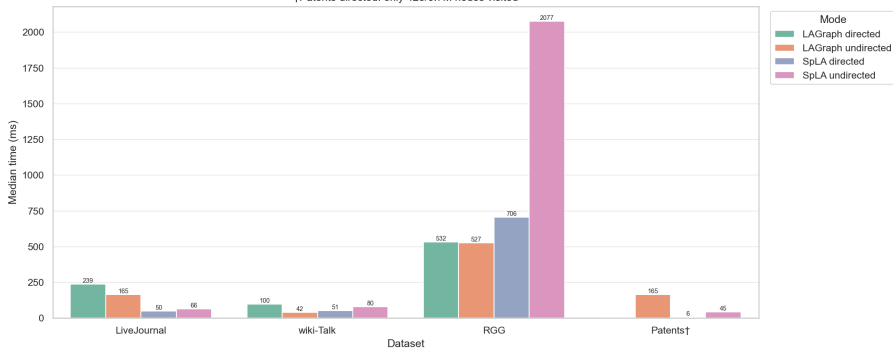
- Без LAGraph_Cached_AT: только push-фаза (медленнее)
- С LAGraph_Cached_AT: direction-optimising BFS (push + pull)

SPLA BFS

--undirected=true/false: симметризует матрицу перед построением графа

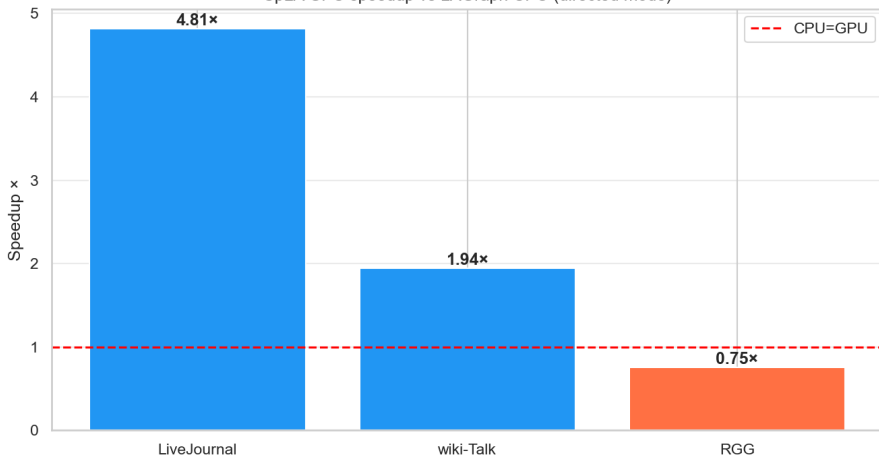
BFS: результаты

BFS median time — all datasets (warmup dropped)
†Patents directed: only 428/3.7M nodes visited



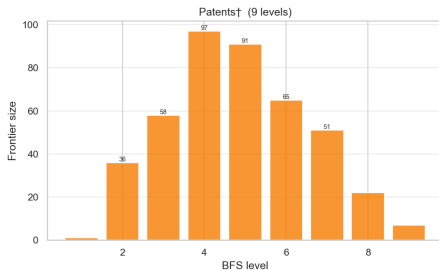
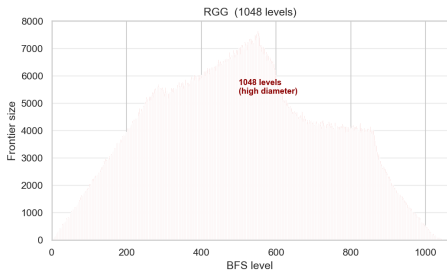
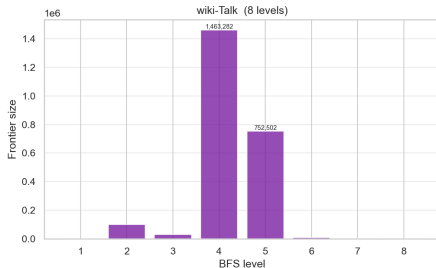
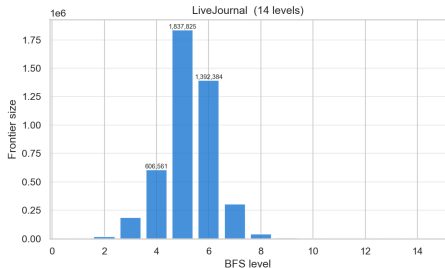
BFS: speedup GPU vs CPU

SpLA GPU speedup vs LAGraph CPU (directed mode)



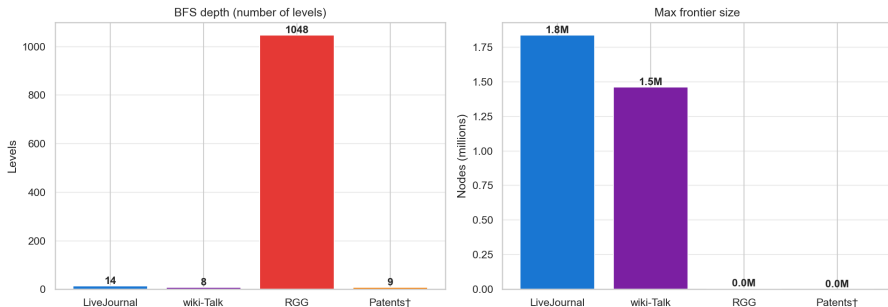
BFS: профилирование — фронтир по уровням

BFS per-level frontier size — topology determines GPU efficiency



BFS: профилирование — итог по топологии

BFS topology: few deep levels = good GPU occupancy



BFS: почему RGG невыгоден для GPU

- RGG — геометрический случайный граф: $\sim 4\text{M}$ вершин, ~ 14 рёбер на вершину
- BFS имеет 1048 уровней
- Максимальный фронт RGG: 7 634 узла против 1.84M у LiveJournal
- Накладные расходы OpenCL
- SpLA directed: 706 мс vs LAGraph CPU: 534 мс — CPU выигрывает на 32%

BFS: выводы

- **SpLA GPU быстрее** на социальных/коммуникационных графах (LJ: $4.8\times$, wiki-Talk: $1.9\times$)
- **CPU быстрее** на RGG — высокий диаметр, малые фронтиры, низкая GPU-загрузка
- **Ненаправленный BFS быстрее** на социальных графах за счёт двунаправленных рёбер в pull-фазе
- BFS memory-bound: 10 потоков CPU (242 мс) медленнее 1 потока (202 мс) (OpenMP overhead)
- patents directed: топологический эффект — 0.01% вершин достижимо вперёд из max-degree-node

SSSP: первичные тезисы

- LAGraph SSSP работает на обоих типах графов
- SPLA SSSP требует ненаправленный граф (push/pull нужна AT)
- GPU эффективен на плотных социальных графах, менее выгоден на разреженных
- $\Delta=1.0$, unit-weights — тестируем накладные расходы реализации

SSSP: поддержка направленных/ненаправленных

LAGraph LAGr_SingleSourceShortestPath

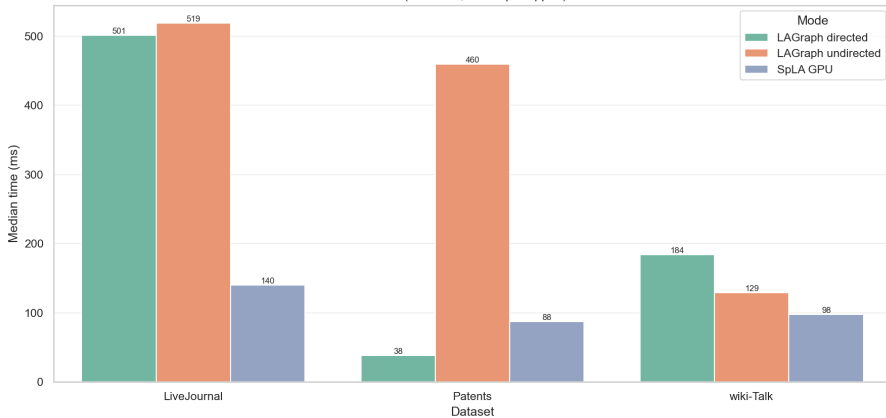
Поддерживает оба режима: directed, undirected

SPLA SSSP: только ненаправленные графы

SPLA SSSP использует push/pull-стратегию аналогично BFS. Для pull-фазы необходима транспонированная матрица AT. При --undirected=false результат может быть некорректным.

SSSP: результаты

SSSP median time (delta=1.0, warmup dropped) — all datasets



SSSP: wiki-Talk — почему SPLA недоступен

SPLA SSSP зависает на wiki-Talk

Граф загружается успешно, вершина-источник выбирается, но вычисление не завершается даже за 5 минут.

Причина: экстремальное распределение степеней

- 74% вершин имеют степень 1 (после симметризации)
- Один хаб: степень 100 029
- Средняя степень: 3.9 — граф очень разреженный

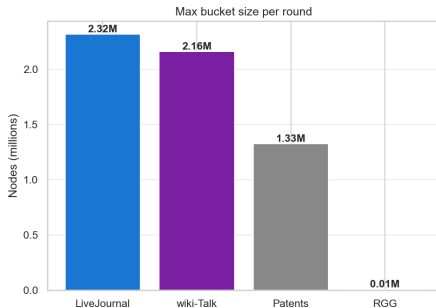
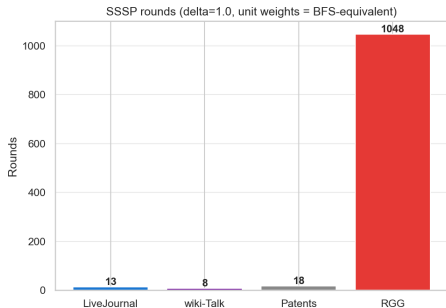
Как это ломает push/pull-переключение:

- После первого раунда фронтير — $\sim 100K$ соседей хаба
- Каждый из них имеет степень $\sim 1-2 \Rightarrow$

LAGraph (CPU) справляется: 180 мс (directed), 126 мс (undirected)

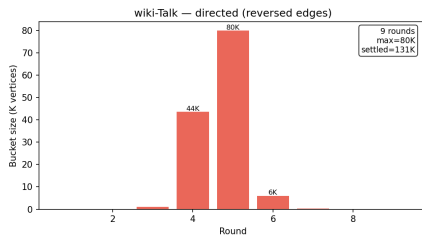
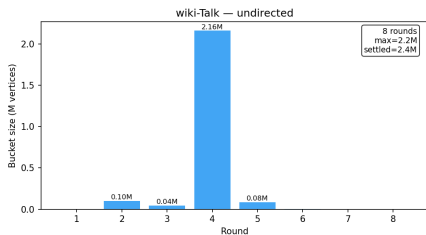
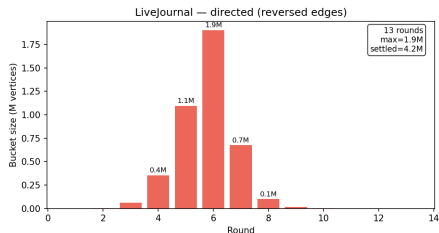
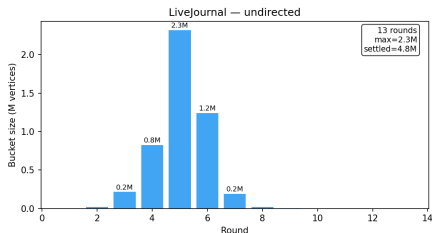
SSSP: профилирование — структура раундов (delta=1.0)

SSSP per-round structure (delta=1.0 → all edges light → same as BFS levels)



SSSP: почему directed vs undirected ведут себя по-разному

SSSP round structure: directed vs undirected (delta=1.0, unit weights)



SSSP: directed vs undirected — объяснение

LiveJournal: directed $\approx$ undirected (+3.5%)

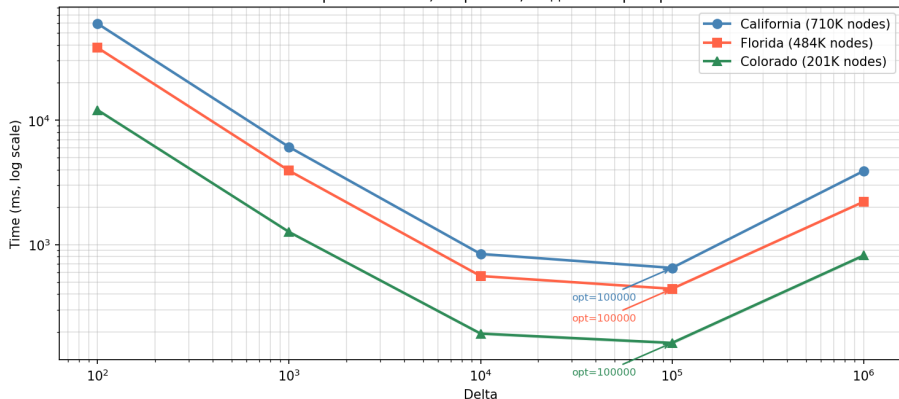
- Социальная сеть — уже почти симметричная
- Симметризация добавляет мало рёбер, новых коротких путей нет
- Одинаковое число раундов (13), но чуть больше nnz в матрице
- Результат: небольшой overhead

wiki-Talk: undirected быстрее на 30%

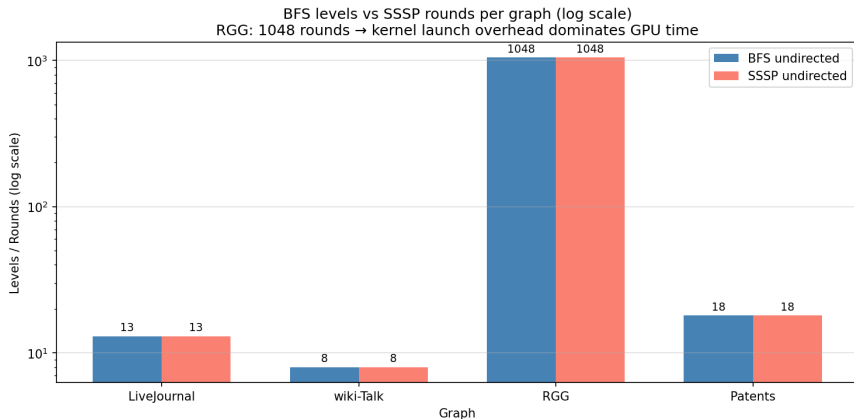
- Граф сильно асимметричный: хаб с 100K рёбер исходящих
- Undirected round 4: 2.16M вершин (90% графа) за один раунд
- Directed: медленное распространение, больше раундов

SSSP: влияние параметра delta (реальные веса, DIMACS10)

SSSP: влияние параметра delta — реальные веса (DIMACS10 census graphs)
LAGraph undirected, 3 прогона, медиана $\pm$ разброс

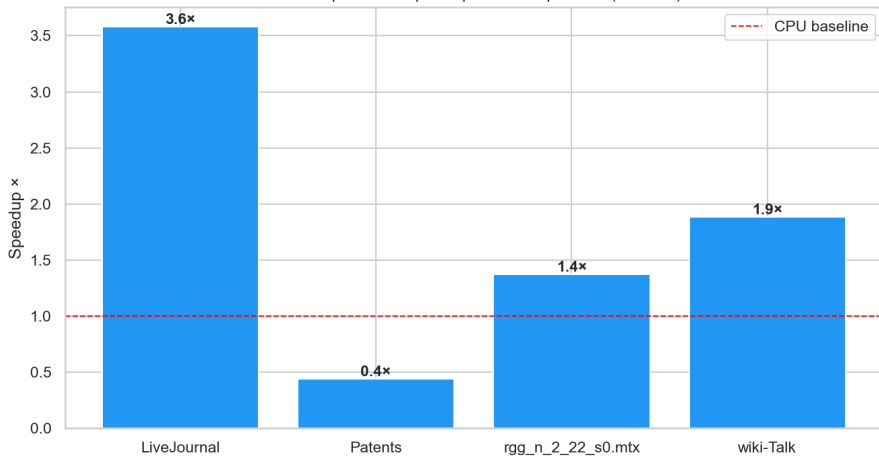


SSSP: BFS-уровни vs SSSP-раунды



SSSP: GPU speedup

SSSP: SpLA GPU speedup vs LAGraph CPU (directed)

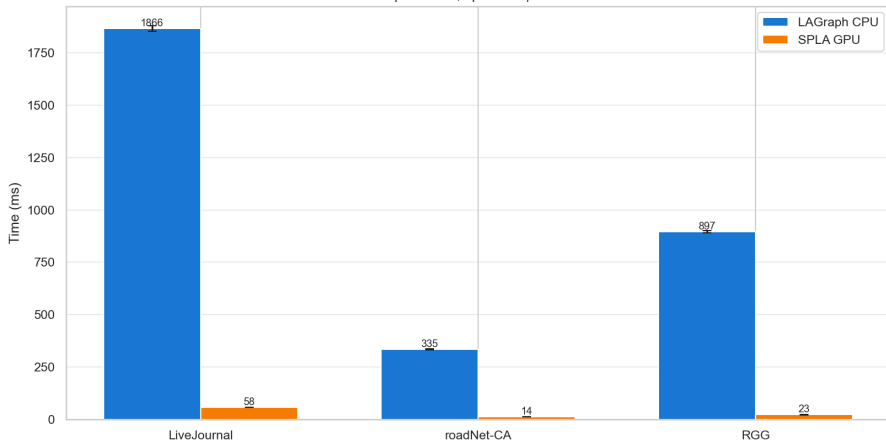


- Delta=1.0 + unit weights: каждое ребро “лёгкое” $\Rightarrow$ delta-stepping вырождается в BFS-подобную обработку — **раунд = BFS-уровень**
- Оптимальное значение delta = 100000
- Ненаправленный SSSP медленнее на patents (+1100%): симметризация добавляет миллионы рёбер для релаксации

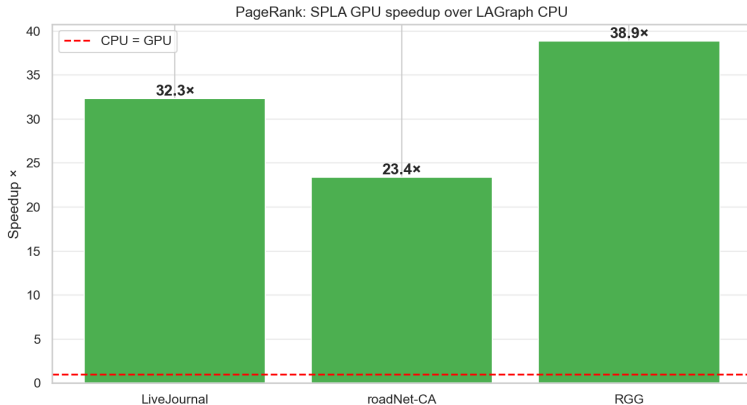
- Основная операция: **SpMV**
- В отличие от BFS: нет разреженного фронта — GPU загружен на каждой итерации
- Гипотеза: GPU покажет многократное ускорение на **всех** графах

PageRank: результаты

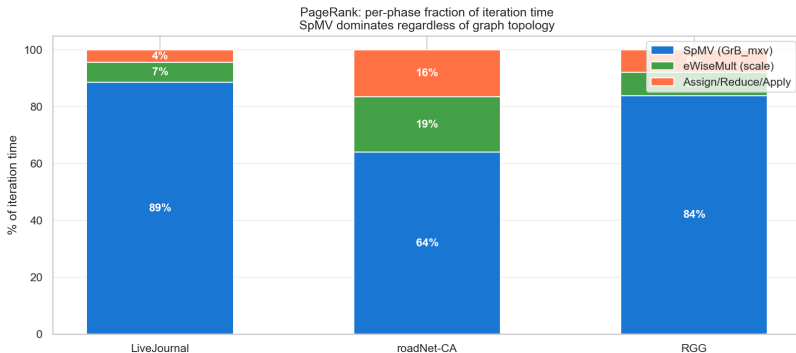
PageRank: total time per graph (warmup dropped, 95% CI)
LAGraph $\alpha=0.85$, $\text{eps}=1e-4$ | SPLA GPU



PageRank: GPU speedup



PageRank: профилирование — доля фаз



- SpLA GPU: **23–39x** быстрее LAGraph CPU на всех трёх графах
- PageRank принципиально отличается от BFS: ранговый вектор плотный, SpMV всегда полностью нагружает GPU
- SpMV = **84–89%** времени итерации на крупных графах (LiveJournal, RGG)
- **Корректность SPLA GPU подтверждена:** `verify_exact` сравнивает GPU-ранки с наивной референсной реализацией поэлементно ($|gpu_i - ref_i| \leq 0.005$, `assert` при несовпадении)

ТС: первичные тезисы

- GPU должен показывать многократное преимущество — masked SpMM идеально ложится на архитектуру
- На разреженных графах (road) GPU менее эффективен из-за накладных расходов
- Предварительная сортировка вершин по степени важна для семейства Sandia на CPU — но **только при правильных условиях**

Семейство Sandia использует разбиение на нижнетреугольную (L) и верхнетреугольную (U) части:

- **Sandia_LL**: $\text{ntri} = \sum L \cdot (L \times L)$ — нижняя $\times$ нижняя
- **Sandia_LUT**: $\text{ntri} = \sum L \cdot (L \times U^T)$ — нижняя $\times$ верхняя^T
- **Burkhardt**: $\text{ntri} = \frac{1}{6} \sum (A^2 \cdot A)$ — полная матрица
- **Cohen**: через пересечение списков смежности

Ключевая операция: для каждого ребра (i, j) в L считаем пересечение $L[i, :] \cap L[j, :]$.

ТС: почему сортировка должна помогать

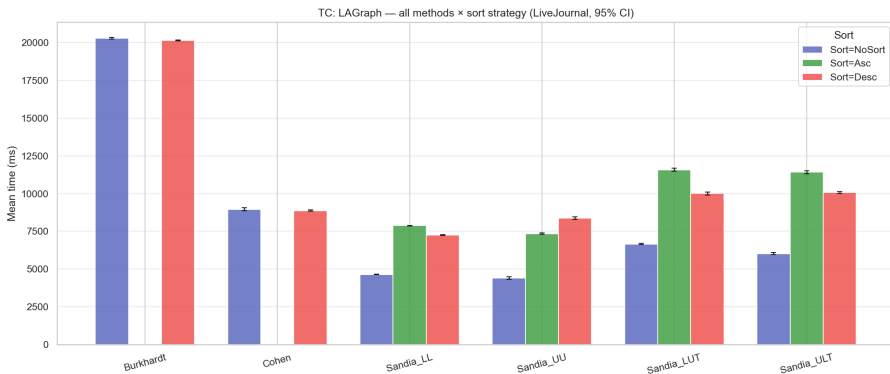
Аргумент стоимости пересечения:

- При сортировке по возрастанию степени: строка i в L имеет $\leq d(i)$ ненулей
- Вершины с малой степенью — в начале матрицы, пересечения их списков дешёвы
- Стоимость пересечения $\approx O(d(i) + d(j))$ — минимизируется когда оба малы

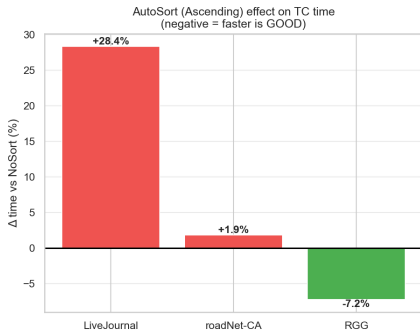
AutoSort LAGraph включает сортировку если выполнены **все** условия:

- $n > 2000$ — граф достаточно большой для амортизации стоимости
- $\text{density} > 10$ — средняя степень достаточно для выгоды
- $\text{mean} > 3 \times \text{median}$ — степенной закон (наличие хабов)

ТС: все методы × стратегии сортировки (LiveJournal)



ТС: эвристика AutoSort — эмпирика



AutoSort heuristic per graph

Graph	Result	n>2000	density>10	mean>3×med
LiveJournal	Yes (+28% slower)	☐ 4.8M	☐ 6.0	☐ 28>27
roadNet-CA	No (no trigger)	☐ 2.0M	☐ 0.003	☐ 2.86≈2.86
RGG	Yes (-7% faster)	☐ 4.2M	☐ 7.2	☐ 14.5>3×14?

ТС: почему сортировка не помогает на наших данных

Профилирование (GxB_BURBLE, soc-LiveJournal1, 20 запусков)

Операция	NoSort	Asc	Desc
mxm	5.71c (25.5%)	5.72c (22.9%)	5.76c (23.1%)
Matrix_extract	—	2.36c (9.4%)	2.44c (9.8%)
SortByDegree	—	0.36c (1.4%)	0.36c (1.5%)
Итого	5.71c	7.33c	7.38c

- Matrix_extract (перестановка $A(P, P)$): +2.36c — предположительно промахи кэша
- Предполагаем сортировка окупается на графах 10× больше, где mxm длится минуты

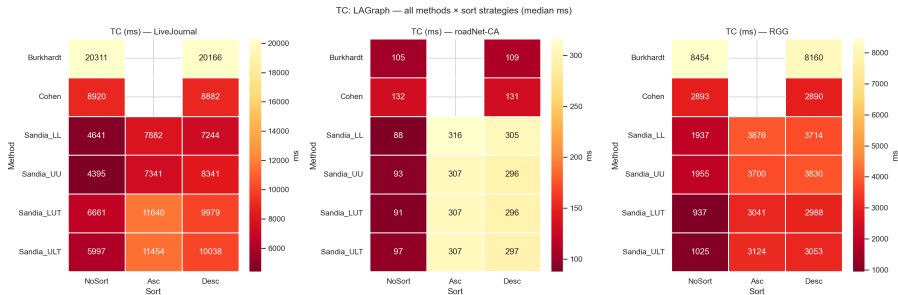
ТС: код сортировки (LAGraph source)

```
if (presort != LAGr_TriangleCount_NoSort) {  
    // Step 1: compute degree permutation P  
    LAGr_SortByDegree(&P, G, true,  
        presort == LAGr_TriangleCount_Ascending, msg);  
  
    // Step 2: permute A -> T = A(P,P)  -- O(nnz), cache-heavy  
    GrB_Matrix_new(&T, GrB_BOOL, n, n);  
    GrB_extract(T, NULL, NULL, A,  
        (GrB_Index*)P, n, (GrB_Index*)P, n, NULL);  
    A = T;  
}
```

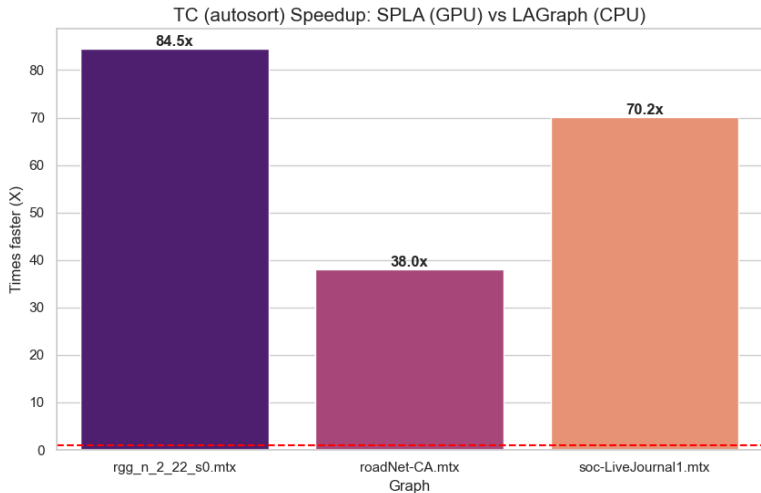
```
// Step 3: L = tril(A), then count = reduce(L .* (L*L))
```

- GrB_extract с индексными массивами = случайная перестановка nnz записей
- Именно эта операция съедает 2.36с при любой сортировке

ТС: все методы × все датасеты



TC: speedup



- SpLA GPU: ускорение до **84.5x** на LiveJournal — masked SpMM идеально параллелен
- **NoSort побеждает** на всех тестируемых графах: overhead перестановки ($\sim 2.7c$) $\gg$ выгода $m \times m$ ($\sim 0.01c$)
- AutoSort корректно определяет условие (LiveJournal: $ratio=3.55 > 3$), но недооценивает стоимость `Matrix_extract`

BFS: direction-optimizing алгоритм (SPLA)

- Push: обходит граф по направлению рёбер (фронтир $\rightarrow$ соседи)
- Pull: обходит граф против направления (непосещённые $\rightarrow$ фронтир через AT)
- Переключение по плотности текущего фронтаира

```
if (push || (push_pull && is_push_better)) {  
    exec_vxm_masked(frontier_new, v, frontier_prev, A, ...);  
} else {  
    exec_mxv_masked(frontier_new, v, A, frontier_prev, ...);  
}
```

- Для Pull на направленных графах необходима AT (LAGraph_Cached_AT)
- Без AT: только push-фаза, алгоритм медленнее на плотных фронтаирах